A Phase Vocoder Approach To Pitch Shifting

Sam Smith

April 28, 2026

1 Introduction

The VST (Visual Studio Technology) and AU (Audio Units) plugin ecosystem greatly contributes to the music industry today. With the rise of digital audio workstations (DAWs) in the early 2000s, technology has become an integral part of music production and composition [10]. In particular, the electric guitar has long been associated with technological advancements in the industry. Engineers first harnessed the power of vacuum tubes to create the earliest electric guitar amplifiers, and later transitioned to transistor-based designs.

Today, traditional amps and effect pedals have been overtaken by virtual amplifiers and software-based effects plugins. Once-dominant traditional hardware companies are now competing with highly technical and lucrative software companies that specialize in music production software [10]. The purpose of this project is to design and implement a high-demand audio plugin for modern music production software.

The plugin in question is a pitch shifter, a widely used effect that digitally modifies an input signal in real time. Its core function is to increase or decrease the input signal's pitch by algorithmically altering every frequency component in a signal [9]. A common problem many guitarists encounter is uptuning and downtuning a guitar. Before detuning a guitar, one must consider a number of physical factors, such as the strings' gauge (thickness), the width of the nut, and the length and tension of the neck. If even one of these factors does not support a major detuning, the guitar cannot be safely or effectively retuned. A pitch shifter removes these physical limitations by solving the tuning problem through a purely software-based approach. The pitch shifter digitally modifies the guitar signal to the desired tuning before it reaches the amplifier. This not only saves time, but also extends the range of the guitar.

Pitch shifting is a desirable signal processing effect used for a variety of practical reasons. Historically, pitch manipulation emerged from research in vocal processing (giving rise to the term 'vocoder' a concatenation of the words 'vocal' and 'coder') [2]. Today, pitch-shifting algorithms are widely used in music production, film, voice acting, and media. Specifically in music production, digital vocal correction tools such as autotuning rely on pitch modification techniques to adjust a singer's intonation while preserving timing and vocal characteristics.

Pitch shifting is not limited to vocal signals, however, it is also commonly applied to instrumental audio. For example, electric guitarists use pitch-shifting pedals and digital audio plugins to alter tuning without physically adjusting string tension. This capability allows musicians to explore alternate tunings without modifying their instrument hardware

or spending vast amounts of time retuning extended ranged guitars. Such applications highlight the broad practical demand for high-quality, real-time pitch-shifting algorithms.

To implement the pitch shifting audio plugin the phase vocoder paradigm will be used. The phase vocoder is a signal processing technique used for both time-scale and pitch modifications. This algorithm was chosen to be the core processing algorithm for this project due to the fact that it is known for its reliability, ability to handle polyphonic signals, and widespread adoption many modern signal processing applications. Furthermore, the phase vocoder is a real-time algorithm; meaning it has the capability to pitch shift an audio signal in a a time frame spanning milliseconds. Hence, the phase vocoder, along with the JUCE audio plugin framework, will provide a basis for creating a real-time pitch-shifting audio plugin.

This paper begins with a comprehensive overview of the mathematics needed to understand the phase vocoder algorithm. It will begin with an explanation of the Discrete Fourier Transform, the backbone of signal processing, then it will examine windowing, why it is need it and what problems it solves. Finally, it will examine the Short-Time Fourier Transform (STFT) which is the core analysis algorithm used in the phase vocoder. The next section will provide a deep intuition for the phase vocoder algorithm where the goal is to connect the mathematical concepts highlighted in the background section to the processing chain that comprises the phase vocoder algorithm. The section will be divided based on the three fundamental steps that make up the phase vocoder algorithm: analysis, modification and synthesis. Next, we will translate the mathematical concepts built up by the previous sections into a coherent program architecture. The audio plugin will be constructed in the JUCE framework, an open-source c++ framework designed to build audio plugins and and applications, Finally, the conclusion follows the results chapter and will primarly focus on results, future work and optimizations that can be done on this project. Additionally, this section will provide insight into any errors or unoptimized processes in this project and offer any additional improvements or next steps.

2 Background

The Discrete Fourier Transform (DFT) is a fundamental tool used in digital signal processing that enables sophisticated analysis and synthesis of discrete time signals [7]. The DFT transforms a signal from the time domain into its frequency domain representation, and does so by expressing it as a sum of sinusoids [7]. The input of the DFT is a finite sequence of N discrete signal samples. The output is N evenly spaced frequency bins and each bin contains a complex number encoding the amplitude and phase for each analysis frequency [8]. Mathematically, the DFT is defined as:

$$X[m] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi nm/N} \tag{1}$$

where N is the number of samples, $x[n]$ is the input signal, n is the time-domain sample index, m is the frequency bin index, and $j = \sqrt{-1}$. The output $X[m]$ is a sequence of N complex values, indexed from $m = 0$ to $m = N - 1$, where each value corresponds to a

frequency bin. Another important property of the DFT is its invertibility. The inverse DFT (IDFT) transforms a signal from the frequency domain to the time domain. It is given by:

$$x[n] = \frac{1}{N} \sum_{m=0}^{N-1} X[m]\, e^{j2\pi n \frac{m}{N}}. \tag{2}$$

The IDFT computes each component of a signal $x[n]$ as the average of the nth samples of the DFT's fundamental sinusoids [8]. It is an important component of the phase vocoder as it transforms a signal back to the time domain after being modified in the frequency domain. In practice, the DFT is much too slow for modern applications. Therefore, applications today use the Fast Fourier Transform (FFT), an optimized algorithm for computing the DFT.

2.1 Windowing

Windowing is a method used to isolate chunks of a discrete time signal so that only a small portion of the signal is analyzed at a time [1]. More specifically, the input signal is multiplied by a windowing function that is nonzero over a finite interval. This results in a small localized snapshot of the input signal where every value outside of the window's central peak is reduced toward zero[12]. Figure 1 below demonstrates the functionality of a window function.

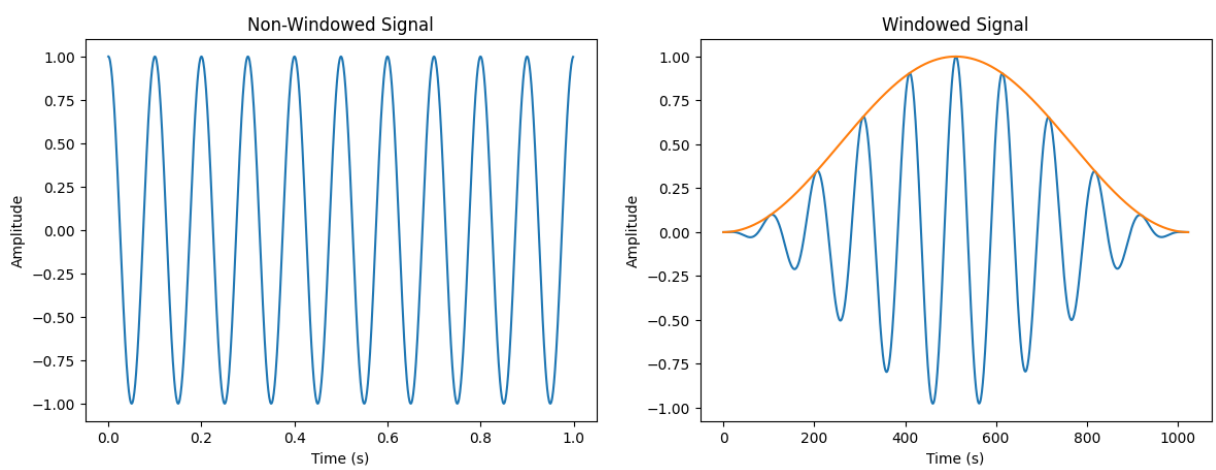

Figure 1: After applying a window function (highlighted by the orange cosine curve), the signal is tapered at the edges, resulting in a temporally localized signal.

As depicted by Figure 1, after applying the window function to the example signal, the sample values near the edges of the signal go toward zero. This property of the window function is an important characteristic as it makes frequency estimations computed by the phase vocoder more accurate. Without windowing, the signal may be discontinuous at its edges: the beginning and end of the signal do not line up.

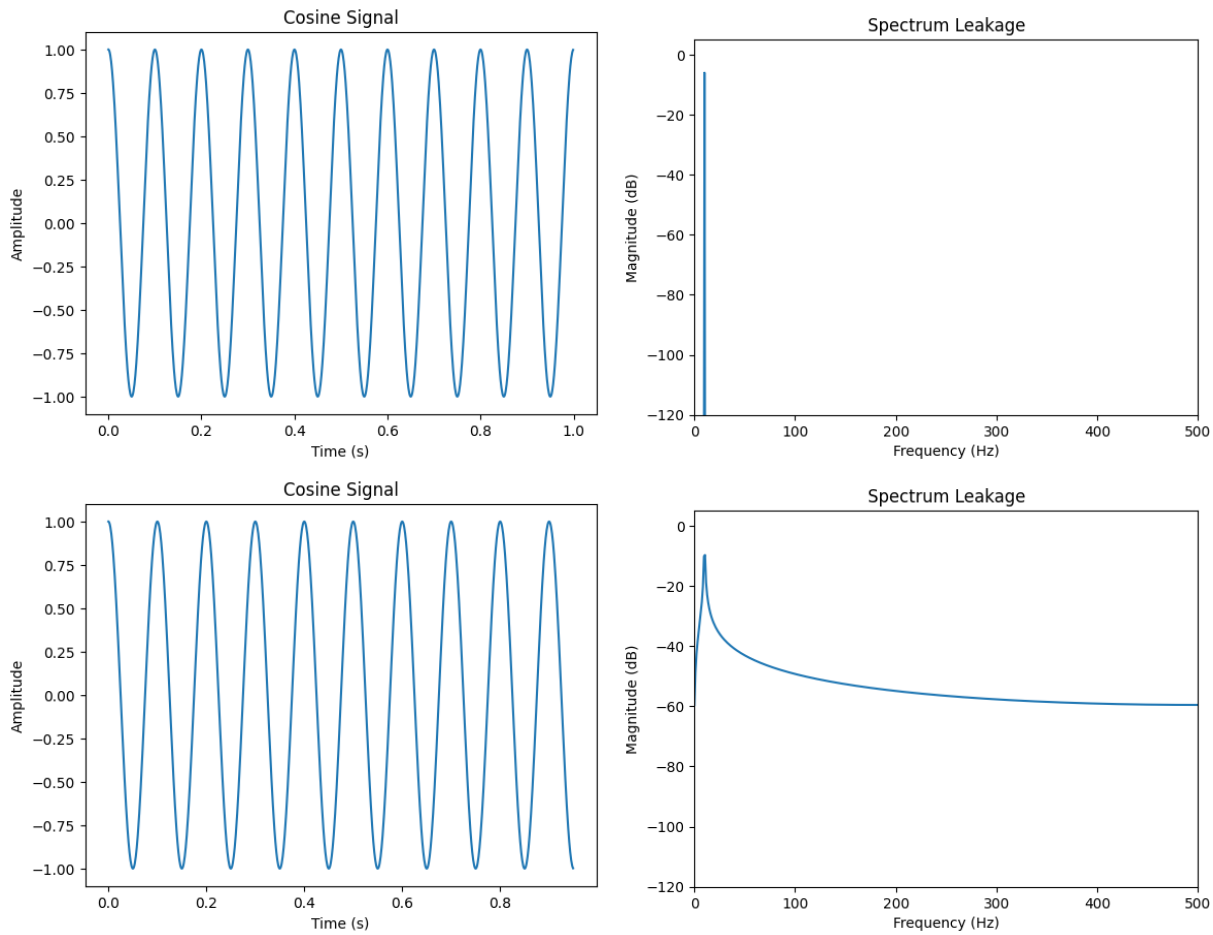

Figure 2: Illustration of spectral leakage. The top row shows a continuous signal segment that begins and ends at the same value, resulting in an accurate frequency representation. The bottom row shows a discontinuous signal segment, where the endpoints do not align, leading to spectral leakage and a spread in the frequency-domain representation.

Figure 2 demonstrates how the same wave can generate different frequency domain representations. The top row shows a continuous signal segment that begins and ends at the same value, resulting in an accurate frequency estimate, as shown by the sharp spectral peak in the corresponding frequency domain plot. When the same wave is segmented and made discontinuous at its edges, as shown by the first graph in the bottom row, it generates spectral leakage, a distribution of the signal's energy across its frequency bins. In order to mitigate the spectral leakage produced by a discontinuous signal, a window function is used to reduce discontinuities at the signal's edges. In Figure 1, the Hanning window function was used to shape the signal, however, there are multiple windowing functions such as the rectangular, triangular, and Hamming window functions [8]. For this project, the Hanning window will be employed as it is commonly used in modern phase vocoder implementations and produces a smoother, more continuous signal than other windowing functions [3] [1].

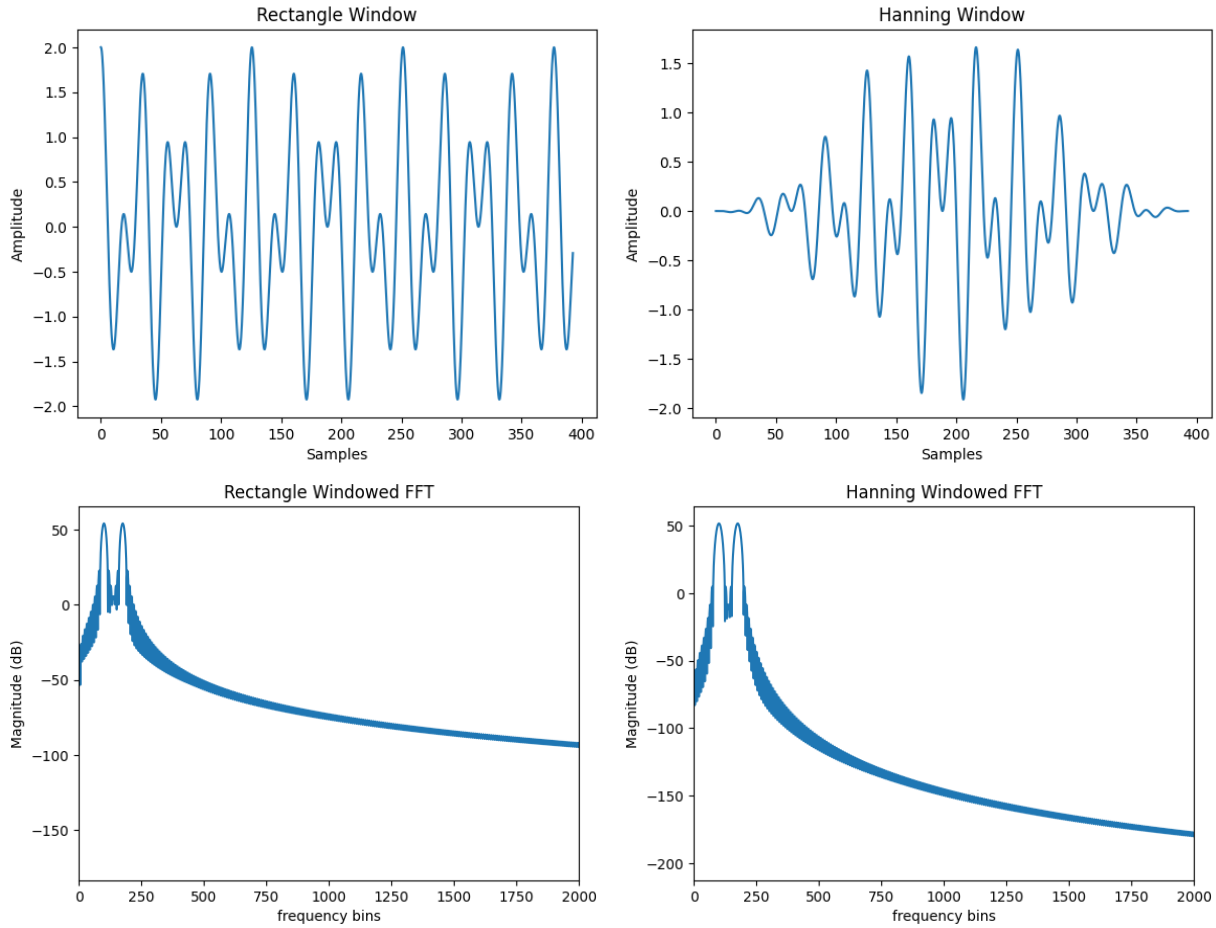

Figure 3: Illustration of the windowing effect. The top row shows the signals after applying a rectangular window and a Hann window. The bottom row shows their corresponding frequency plots. Applying the Hanning window reduces spectral leakage, making the main frequency component more accurately represented.

In Figure 3 that the Hanning frequency plot (second row) tapers off much more rapidly than the rectangular spectrum. The regions of a frequency spectrum outside of the main peaks are referred to as sidelobes. If the sidelobes are too prominent in the frequency representation, frequency estimations will be less accurate. Hence, a windowing function is used by the phase vocoder to reduce spectral leakage and compute more accurate frequency estimations.

2.2 Short-Time Fourier Transform

The Short-Time Fourier Transform (STFT) is a time-frequency representation used to compute the DFT of overlapping windowed segments of a signal. In practice, however, the STFT uses the FFT as this is more computationally efficient. The technique multiplies the input signal by a windowing function so that only a small portion of the signal is analyzed [2]. In order to move the window and isolate different chunks of the input signal, the samples are offset by

a hop size. The hop size is determined by how much the successive STFTs should overlap. The STFT is given by:

$$X[m,k] = \sum_{n=0}^{N-1} x[n+kH]w[n]e^{-2\pi jmn/N}. \tag{3}$$

The output of the STFT, $X[m,k]$, is a two dimensional array indexed by both frequency bins m and frame index k. The section of the input signal is given by $x[n+kH]$, where kH determines the position of isolated window frame.

In practice, typical hop sizes are 75%, 50%, 33.3%, and 20% overlap. The amount of overlap introduces a tradeoff between time resolution and frequency resolution [12]. Lastly, the STFT is also invertible. However, instead of reconstructing the entire signal all at once, the inverse STFT (ISTFT) individually converts the signal frames back to the time domain . This is especially useful in phase vocoder based pitch shifter, as a signal can be modified in the frequency domain and then subsequently transformed back to the time domain.

3 Related Works

Pitch-shifting has been approached by a number of time–frequency methods. Early methods, such as Dolson's (1986), employed the use of the Short-Time Fourier Transform in conjunction with the phase vocoder algorithm. In particular, Dolson describes a pitch-shifting method that relies on time-scale manipulation to modify the pitch of a given signal [2]. For example, by stretching a signal, making it longer in the time domain, and then rescaling it back to its original length, the overall frequency content of the signal is effectively decreased, which corresponds to lowering the signal's pitch. Dolson acknowledges, however, that this approach is imperfect: specific constraints must be enforced when performing a pitch shift, as failing to do so may introduce unwanted artifacts into the signal or result in a reconstructed signal that has lost information [2]. While effective for monophonic (single note) pitch scaling, this classical time-scale modification framework treats spectral bins independently, which can lead to phase incoherence and unwanted artifacts. These limitations motivated subsequent refinements to the phase vocoder algorithm.

Unlike the original process chain described by Dolson (1986), which applies a uniform scaling factor to all spectral components, later work by Laroche and Dolson (1999) introduces refinements that improve frequency coherence [5]. In particular, they highlight a new peak-detection paradigm, where peaks refer to how much a particular frequency is in the input signal. Once the peaks are detected, they can be shifted around by a pitch-factor and unlike the version described by Dolson (1986), the pitch-factor is not restricted to an integer amount [5].

In addition to optimizing frequency resolution, modern implementations of the phase vocoder shift away from the standard two-step timescale and resample approach and adopt an analysis and resynthesis paradigm [3]. The new approach leverages phase information obtained from the STFT step to more accurately reconstruct the modified signal [3]. More specifically, a phase adjustment value can be derived from the frequency bins across two

frames and when added to the original frequency approximation the accuracy of the frequency estimation is increased [6].

The prior sections highlight the phase vocoder approach to pitch-shifting, a frequency-domain algorithm that converts a signal from the time domain to the frequency domain to compute the pitch shift. An alternative approach uses time-scale modification (TSM) algorithms, which operate entirely in the time domain [9]. A common algorithm belonging to the TSM family is the Overlap-Add (OLA) algorithm [12].

The idea behind these algorithms is that pitch shifting and time scaling are essentially the same; meaning if you have one, either a pitch shifter or time scaler, then you effectively have the other. OLA makes use of this paradigm. The algorithm first partitions the input signal into frames, much like the STFT. Then, if the goal is to pitch down the signal, when resynthesizing the signal it is constructed in such a way that extra space is added between the reconstructed frames, increasing the length of the signal. On the contrary, if the goal is to make the signal's pitch higher, then the algorithm reconstructs the signal in such a way where the space between contiguous frames is reduced, effectively shortening the signal's length [12]. Since TSM algorithms operate in the time domain and thus change the input signal's duration, either lengthening or shortening it, these algorithms must incorporate a resample step in order to convert the modified signal back to the original signal's duration [9]. Thus, these alternative algorithms are viable options; however, purely TSM-based algorithms are often outperformed by frequency-based algorithms [12].

4 Phase Vocoder

The phase vocoder is the name given to a group of algorithms designed to more accurately analyze and synthesize an input signal; it uses the signal's phase information to obtain a more precise estimate of the signal's frequency [11]. This paper specifically focuses on the application of pitch shifting, the process of either increasing or decreasing an audio signal's pitch. In particular, this work utilizes a phase vocoder-based pitch shifting algorithm, as it is a computationally efficient method for analyzing and modifying an input audio signal, and is the backbone of many real-time pitch shifting audio plugins used today [12].

The following subsections will explain the inner workings of the phase vocoder algorithm and its application to pitch shifting. The first subsection will explain the analysis step and how the STFT breaks an audio signal into frames. The second subsection will cover the modification step and explain how pitch shifting is computed in the frequency domain. Finally, the last subsection will explain how the modified signal is synthesized back into an output signal.

4.1 Analysis

The first step of the phase vocoder algorithm is the analysis step. The primary goal of this step is to segment an input audio signal into a succession of overlapping frames. This way, each frame can be processed individually. In a real-time setting, the phase vocoder does not have access to the entire signal at once. The signal is being continuously fed into the phase

vocoder from a source. Thus, the phase vocoder is not operates locally, processing the signal frame by frame.

In practice, the STFT is used to partition the audio signal into frames of N samples. This is accomplished by multiplying the signal frame by the Hanning window function. Once the signal frame has been isolated, the FFT is computed, converting the frame from the time domain to the frequency domain. To process the next frame, the analysis window moves ahead by H samples, where H is the hop size. Figure 4 below highlights the individual operations of the analysis step.

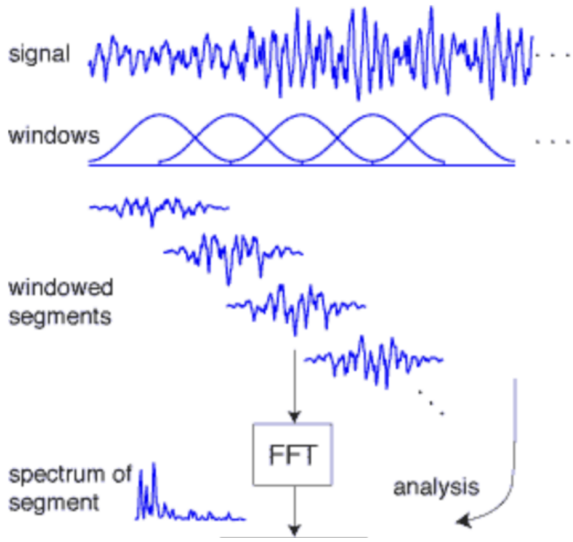

Figure 4: Analysis step illustration from [13]

The output of the analysis step is a collection of M frequency bins (see Section 2.2), where each bin contains a complex value representing the magnitude and phase information for a particular frequency [11]. At this point, the signal frame can be modified in the frequency domain, such as pitch shifting.

4.2 Modification

In this section, I will explain how the phase vocoder is used to modify an audio signal. With the completion of the analysis step, the audio signal has been transformed into the frequency domain. The signal is now represented as linearly spaced frequency bins that correspond to frequencies ranging from 0 to $f_s/2$, the Nyquist frequency. The bins above the Nyquist frequency are not included as they contain redundant information. Each bin k is a complex number of the form $a + jb$ that represents a specific analysis frequency [8]. An analysis frequency can be thought of as sinusoid that completes an integer number of cycles over an N sample window. Thus, each frequency bin k measures the contribution of a sinusoid at frequency $\frac{k}{N} * f_s$, where f_s is the sampling rate [8].

One shortcoming of the FFT, however, is that a signal's fundamental frequencies may fall between frequency bins. For example, a typical sampling rate might be 44100Hz, and let

our window size be 1024 samples. To calculate the spacing between frequences we compute:

$$\Delta f = \frac{f_s}{N} = \frac{44100}{1024} \approx 43\text{Hz}$$

Therefore, at bin 0 the analysis frequency is 0Hz, at bin 1 it is 43Hz at bin 2 it is 86Hz and so on. If a signal's frequency falls between one of these bins, its energy is distributed accross multiple nearby bins and the actual frequency is unable to be represented by a single bin [6]. The energy distribution manifests itself as spectral leakage and as seen in Section 2.1 a windowing function helps to mitigate this problem.

4.2.1 Computing a More Accurate Frequency

To more accurately calculate a signal frame's frequency, the phase vocoder employs the use of the frame's phase information from the FFT. The core principle behind this computation is that frequency is the derivative of phase. Thus, by using a signal's phase information from two different hops the exact frequency of the signal can be calculated. The equation below mathematically demonstrates the frequency of a signal ω written as a first order derivative in terms of phase ϕ:

$$\omega[n] = \frac{\phi[n] - \phi[n-H]}{H}. \tag{4}$$

Now, we can rearrange Equation 4 to isolate the phase difference:

$$\phi[n] - \phi[n-H] = \omega[n] \cdot H = \frac{2\pi k H}{N}. \tag{5}$$

Thus, the phase shift for a sinusoid with a frequency exactly matching a frequency bin over one hop size is given by Equation 5. What we now have is a basis to compare the actual signal frequency to. Computing the adjustment factor, also referred to as the phase remainder ϕ_r, we can subtract the ideal phase from the actual observed phase [6]:

$$\phi_r[n] = (\phi[n] - \phi[n-H]) - \frac{2\pi k H}{N}. \tag{6}$$

In order to guarantee that the phase vocoder computes the smallest phase deviation, we must wrap ϕ_r between $(-\pi, \pi]$. Once the phase is wrapped, we can calculate the deviation of the observed frequency from the bin frequency:

$$\omega[n] - \omega_k = \frac{\text{wrap}(\phi_r[n])}{H}, \text{ and} \tag{7}$$

$$\omega[n] = \frac{\text{wrap}(\phi_r[n])}{H} + \omega_k \tag{8}$$

where ω_k is the frequency of the kth bin. We now have an equation that computes the exact frequency of the sinusoid. Due to operating in a real-time environment, however, it is more computationally efficient to compute a fractional FFT bin number; a way to represent a frequency in terms of where it would fall if the bins had infinite resolution. To compute

the fractional FFT bin number, we only need to scale Equation 9 by a factor of $2\pi/N$ since $\omega[n] = \frac{2\pi k}{N}$. Doing so yields:

$$b[n] = \frac{\text{wrap}(\phi_r[n])N}{2\pi H} + k \tag{9}$$

and we now have a method to more accurately compute the frequency of our signal.

4.2.2 Pitch Shifting and Synthesizing a Signal

Now that we have calculated a more accurate frequency estimation, pitch shifting the signal involves scaling the frequency $b[n]$ by a pitch shift factor R. In doing so, we compute a new fractional bin $b_s[n]$, where $b_s[n] = Rb[n]$. After that, we calculate the new bin number rounded to the nearest integer: $k' = \text{floor}(Rk + 0.5)$ which will contain the shifted frequency. The final part of the modification step is calculating a new phase value for the shifted frequency. The new phase value ensures the modified signal is reconstruced correctly and does not introduce any discontinuities at the signal frame's edges [1]. The calculation is similar to Equation 9 with a few adjustments. To compute the new phase value, we are only concerned with how much the phase has changed from one hop to the next. Thus we first compute the phase remainder of the new frequency:

$$\phi_{rs}[n] = \frac{2\pi H(b_s[n] - k')}{N}. \tag{10}$$

Next, we add the ϕ_{rs} to the phase of the shifted signal that perfectly fits into a frequency bin to obtain:

$$\frac{2\pi H b_s[n] - 2\pi H k'}{N} + \frac{2\pi k' H}{N} = \frac{2\pi H b_s[n]}{N}. \tag{11}$$

Recall that $b_s[n]$ is the fractional bin index, hence the result of Equation 11 is the phase of the signal frame's frequency estimation. Finally, by adding this frame's phase to the previous hop's phase: $\phi_s[n - H]$, where $\phi_s[n - H]$ is the phase of the same bin at the previous hop, we compute the actual phase of the current bin at the current hop. This ensures that the phase evolves continuously across frames. This is mathematically defined as:

$$\phi_s[n] = \text{wrap}\left(\phi_s[n - H] + \frac{2\pi H b_s[n]}{N}\right) \tag{12}$$

where $\phi_s[n]$ is the actual phase of this bin at the current hop.

4.3 Synthesis

Once the signal frame has been modified and a new phase value has been calculated, the IFFT converts the signal back to the time domain. The resulting signal segment is then reconstructed using the Constant Overlap-Add (COLA) paradigm. A detailed explanation of COLA can be found here [14]. The principal idea is that if samples in the window function, offset by H, add to a constant C, then the window possesses the "Constant Overlap-Add

Property" [14]. The COLA property is defined mathematically as:

$$\sum_{m=-\infty}^{\infty} w(n - mH) = 1, \quad \forall n \in \mathbb{Z} \tag{13}$$

and Figure 5 below graphically shows the COLA Property.

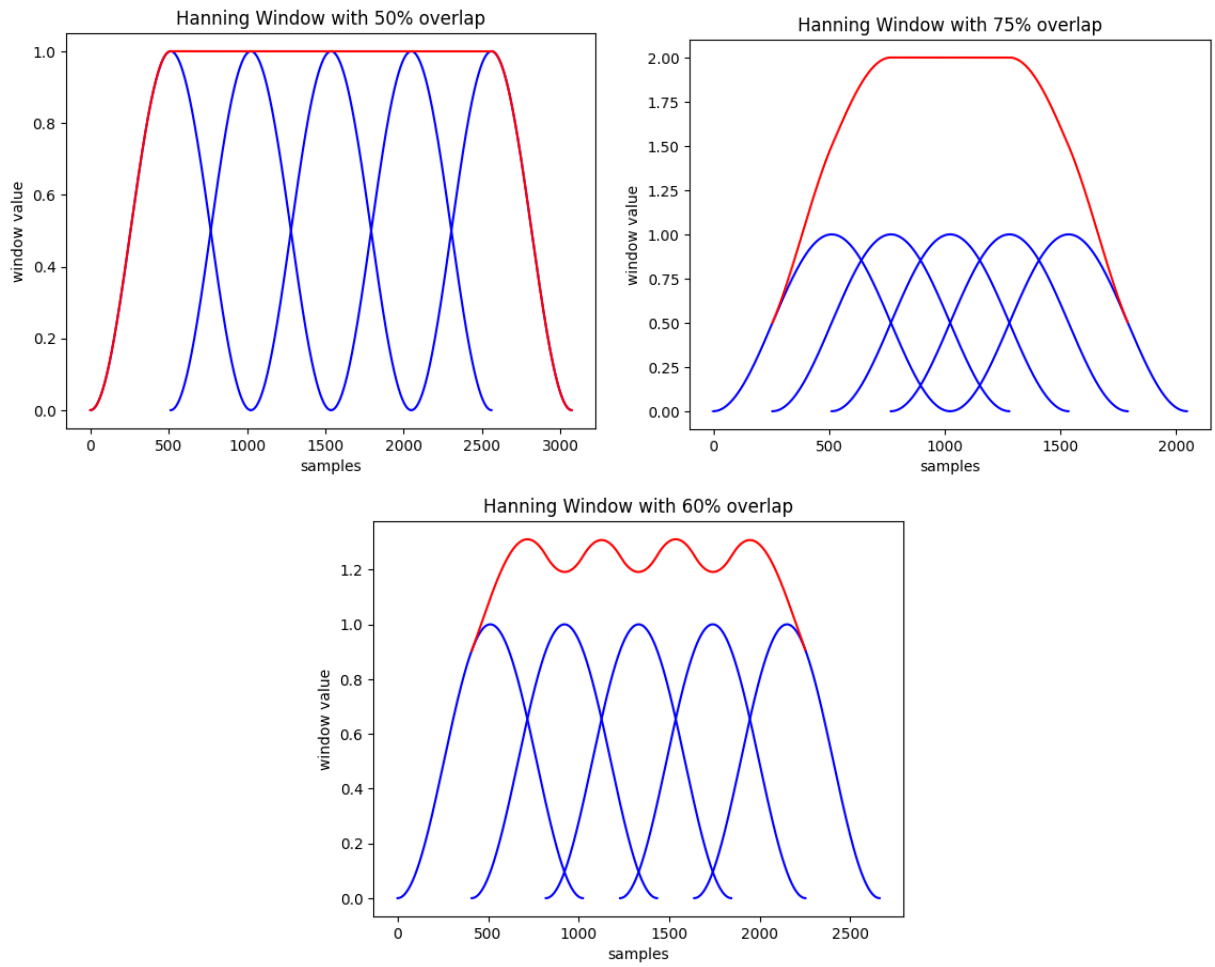

Figure 5: Illustration of the Constant Overlap-Add (COLA) property. The first graph demonstrates that a 50% overlap satisfies the COLA property. The second graph uses a 75% overlap which does posses the COLA property. The third graph uses a 60% overlap which does not satisfy the COLA property

The first plot in Figure 5 demonstrates that the Hanning window with a 50% overlap possesses the COLA property. The red line represents the sum of the Hanning window samples offset by a hop size of 512 samples. Once the five Hanning windows start to overlap, their samples sum to a constant value. The last plot (second row) demonstrates that a 60% overlap does not posses the COLA property. Notice that once the Hanning windows overlap, the samples do not sum to a constant value. This project employs the Hanning window

function with a 75% overlap. Even though this amount of overlap does not yield perfect resynthesis, it is regarded as the standard amount of overlap for modern phase vocoders [1]. As seen in section 4.3, the phase vocoder uses previous phase values to more accurately estimate a signal's frequency. Thus, by having a larger overlap, the phase vocoder more frequently analyzes the input signal which results in more accurate phase calculations [1].

Figure 6 below demonstrates the synthesis step.

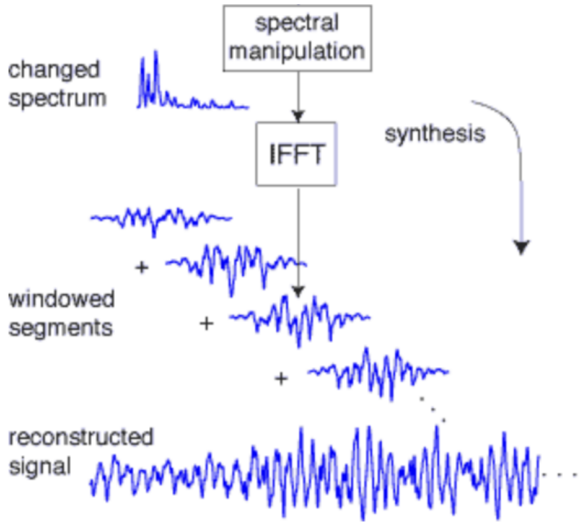

Figure 6: Illustration of the synthesis step from [13]

As seen in Figure 6, the time domain frames are recombined using the overlap-add technique to produce a final signal. To reconstruct the output signal, the time domain samples from the ISTFT are first added into an output buffer. This buffer serves as temporary storage; it is not the final synthesized audio signal. The output buffer is necessary because, due to overlapping windows ensured by the COLA property, certain samples are calculated as the sum of multiple windows. Once the samples have been correctly added in the buffer, a write pointer is used to sequentially read them to the output audio buffer.

5 Methodology

The JUCE framework is used to implement the pitch-shifting audio plugin. JUCE is an open-source framework that supports the development of cross-platform audio applications and plugins. Its integrated build system allows projects to target multiple plugin formats, such as VST3, AU and more. JUCE projects are written in C++, making it well-suited for developing highly efficient, real-time audio plugins. Additionally, JUCE provides a wide range of built-in libraries that support common audio processing tasks, as well as tools for creating graphical user interfaces (GUIs).

Under JUCE's framework, the pitch shifter processes contiguous blocks of an input signal. The signal is passed to the pitch shifter using JUCE's PluginProcessor.ProcessBlock() method. In this method, the audio buffer (block of signal) is divided into its respective

channels. Then for each channel, its audio samples (discrete data values) are passed to the pitch shifter for further processing. Once in the scope of the pitch shifter, each sample in the channel data is stored in an input buffer. Once this buffer has 1024 samples, the buffer will be windowed and the FFT will be computed to obtain the signal's magnitude and phase information across frequency bins. To pitch shift the more accurate frequency estimation, a pitch factor is used to scale the frequency components of the signal. Once modified the signal is reconstructed by using prior phase information to enforce continuity across frames, and the inverse STFT is computed to convert the signal back to the time domain. JUCE then takes the modified sample data and passes it back to the DAW where it can then be sent to speakers, or additional plugins for further processing.

To test the audio plugin's shifting capabilities, it was used in an offline and real-time environment. In the offline environment, a pre-recorded guitar track was uploaded to a Reaper, the DAW, where the pitch shifter was selected as the only plugin for the track. Throughout the track's duration, the plugin was used to shift the audio track up to an octave in either direction. To test the audio plugin in a real-time environment, an electric guitar and guitar amplifier were connected to Reaper via an audio interface and the pitch shifter was loaded as an effect in Reaper. The guitar signal first passed into Reaper where the pitch shift effect was applied, then out to the amplifier where the sound was played.

6 Results

To accommodate the pitch shifter audio plugin, a simple dial was constructed using the JUCE framework's built in GUI library. The dial allows the user to control the pitch shift range up to 12 semitones in either direction (pitch up or pitch down). Figure 7 below is the audio plugin loaded into a DAW.

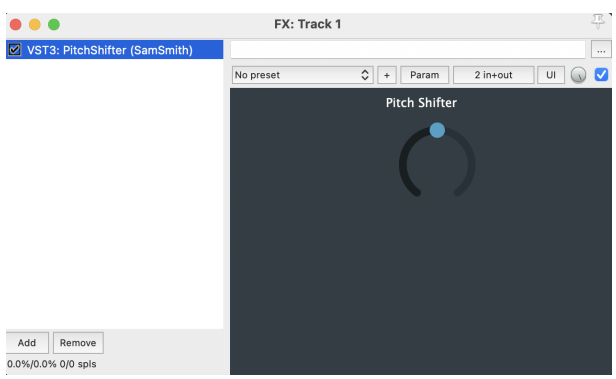

Figure 7: Illustration of the audio plugin dial

On hovering the mouse over the dial, the amount of shift is displayed to the user. The range of the dial is set to [-12, 12] inclusive, which allows for the signal to be shifted by an octave in either direction.

Results from the offline tests indicated that the audio track had no audible loss of resolution or clarity when the signal was shifted. Shifting the track up and down did not

introduce any additional artifacts, which typically manifest as clicks and pops in the audio. Due to time restrictions, however, a more thorough analysis of the signal was not completed. To test the audio plugin in the real-time environment, both a high gain and clean guitar tone were used in a real-time setting. Each effect was also accompanied by additional effects, such as reverb and delay, which were applied to the signal by the amp, after Reaper applied the pitch shifter. The results indicate a drop in audio quality from the offline tests. The phasiness effect, a smearing of the audio signal [4], was more noticeable in the high gain test. Additionally, this particular implementation is better at shifting monophonic signals, one note at a time, than polyphonic signals (chords). This limitation was noticeable in the clean guitar test, where two notes are played at the same time resulting in an audible drop in sound quality. Similarly to the offline tests, more sophisticated techniques were not employed to measure the quality of the signal due to time constraints. Thus, moving forward, techniques from [9] could be implemented to analyze the quality of the shifted signal.

- A successful pitch-shift audio plugin will produce a shifted audio signal with minimal artifacts introduced
- Additionally, the success will depend on how well it can handle multiple notes. Full chords, two power chords, fast-alternate picking etc.
- If time permits, I will test the audio clarity using one of the tests found here: [9]

7 Conclusion

This project effectively demonstrates a working implementation of a phase vocoder based ptich shifter. It is able to both pitch up and down an octave (12 semitones), while maintaining reasonable audio clarity and introducing minimal artifacts. The solution follows the phase vocoder paradigm: converting the signal from the time domain to the frequency domain, using phase information obtained from this step to improve frequency estimations, modifying the signal in the frequency domain, and finally converting the signal back to the time domain.

While this implementation is good at shifting monophonic signals, it struggles at accurately pitch shifting polyphonic signals (multiple notes at once, chords). More advanced pitch shifters employ techniques such as peak-locking, zero-padding and advanced phase techniques [12]. One notable future improvement would be to examine alternative methods of phase reconstruction that result in a more complete signal after signal modification.

- need some closing sentence

References

[1] Barry, D., Dorran, D., and Coyle, E. Time and pitch scale modification: A real-time framework and tutorial.

[2] Dolson, M. The phase vocoder: A tutorial. *Computer Music Journal 10*, 4 (1986), 14–27.

[3] Götzen, A. D., Bernardini, N., and Arfib, D. Traditional (?) implementations of a phase-vocoder: The tricks of the trade. In *Proceedings of the COST G-6 Conference on Digital Audio Effects (DAFx-00)* (Verona, Italy, Dec. 2000), pp. 37–44.

[4] Juillerat, N., and Hirsbrunner, B. Low latency audio pitch shifting in the frequency domain. In *2010 International Conference on Audio, Language and Image Processing (ICALIP)* (2010), IEEE, pp. 1–6.

[5] Laroche, J., and Dolson, M. New phase-vocoder techniques for pitch-shifting, harmonizing and other exotic effects. In *Proceedings of the IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)* (New Paltz, NY, USA, Oct. 1999), IEEE, pp. 91–94.

[6] Lim, K. A. An open-source phase vocoder with some novel visualizations. Capstone project, School of Informatics, Indiana University Bloomington, June 2007.

[7] Lyons, R. G. *Understanding Digital Signal Processing*, 3rd ed. Pearson, Upper Saddle River, NJ, USA, 2010.

[8] McFee, B. Digital signals theory, 2025.

[9] Rai, A., and Barkana, B. D. Analysis of three pitch-shifting algorithms for different musical instruments. In *2019 IEEE Long Island Systems, Applications and Technology Conference (LISAT)* (2019), IEEE, pp. 1–6.

[10] RaNa, M. R. H. The influence of technology on modern music production. *International Journal of Humanities and Information Technology 6*, 04 (2024), 19–25.

[11] Reiss, J. D., and McPherson, A. *Audio Effects: Theory, Implementation and Application*. CRC Press, 2014. Accessed as PDF.

[12] Royer, T. Pitch-shifting algorithm design and applications in music. Master's thesis, KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2019.

[13] Sethares, W. A. A phase vocoder in matlab, n.d.

[14] Smith, J. O. *Spectral Audio Signal Processing*. 2011. Online book.